UNIVERSITÄT TÜBINGEN
PROF. DR.-ING. HENDRIK P.A. LENSCH
LEHRSTUHL FÜR COMPUTERGRAFIK
PHILIPP LANGSTEINER (PHILIPP.LANGSTEINER@UNI-TUEBINGEN.DE)
MEHRAN HOSSEINZADEH (MEHRAN.HOSSEINZADEH@UNI-TUEBINGEN.DE)
DEBORAH KORNWOLF (DEBORAH.KORNWOLF@STUDENT.UNI-TUEBINGEN.DE)

FEBRUARY 19, 2026

MASSIVELY PARALLEL COMPUTING
ASSIGNMENT 0

Since we're running the exercises online, there are a few setup steps you need to go through first before starting with the actual exercises. Please let us know if you have any questions or suggestions.

In principle, you can also run the exercises on your own machine if you have a CUDA-capable graphics card (i.e., one from NVIDIA). However, we can *not* support your individual environments on your machines.

0.1 Sign Up

Please fill in the "Student Information" survey on Ilias. If you do not fill this survey we can't grade your exercises and exam.

0.2 Basic Setup

Please follow the instructions in the Computer Graphics Getting Started Guide to setup a SSH and VNC connection to our machines. Make sure to setup both, don't forget the VNC connection. We will check your setup on the first day of the course to ensure a smooth exercise experience. We are there that day to help you with any problems that might occur. Please note that demonstrating your setup that day is mandatory to participate in the remaining exercises.

0.3 CUDA Programming Environment

In order to compile and run CUDA applications, you need the CUDA Toolkit.

To test your CUDA environment, try running the NVIDIA CUDA Compiler `nvcc`:

```
$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Mar_28_02:18:24_PDT_2024
Cuda compilation tools, release 12.4, V12.4.131
Build cuda_12.4.r12.4/compiler.34097967_0
```

Also check your NVIDIA driver and CUDA Toolkit versions using `nvidia-smi`:

```
$ nvidia-smi
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.120                Driver Version: 550.120        CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
...
```

0.3.1 Choosing an IDE

For developing CUDA applications, you can choose between a few IDEs:

- Visual Studio Code with the Nsight Visual Studio Code Edition extension in addition to the usual C/C++ setup (C/C++, clangd, CMake Tools, Error Lens extensions).

  Debug configurations need to be created manually as `launch.json` files. Make sure to set `"cwd"` to the absolute path of the project, `"program"` to the absolute path of the compiled program, and `"args"` to the program arguments. You can start with a copy of this example:

  ```
  {
      "version": "0.2.0",
      "configurations": [
          {
              "name": "Example Debug Configuration",
              "type": "cuda-gdb",
              "request": "launch",
              "cwd": "${workspaceFolder}",
              "program": "${workspaceFolder}/build/testGamma",
              "args": "vase.ppm 2.2 vase_gamma.ppm"
          }
      ]
  }
  ```

  (See the Variable Reference)

  While it is possible to debug CUDA threads, switching between them requires entering the thread and block ids manually and there is no overview of running threads or blocks like in Eclipse.

- Eclipse IDE for C/C++ Developers with the Nsight plugins.

  Eclipse is not widely used for C++ development (anymore), but used to be the basis of NVIDIA's official Nsight IDE (now deprecated). Thus, it has the best integrated debugger for CUDA which provides a nice overview of all running CUDA threads and allows to easily switch between them.

  CMake support is bad (system includes are not being found and, thus, code indexing fails). Instead, it depends on its own build system or manually written `Makefile`s.

  Make sure to *exactly* follow the steps shown below on how to open/create CUDA projects in Eclipse.

  When installing Eclipse version 2022-12 or later, make sure to use the plugins from CUDA Toolkit version 12.3 or later. Otherwise, you will not be able to run the debugger.

- Kate

  Sometimes, a simple text editor is all you need. It has support for `clangd` (you might have to enable the LSP plugin in the settings), so you get the full C++ code model, syntax highlighting, completion, etc.. If you're looking for something small, this might be the right tool for you.

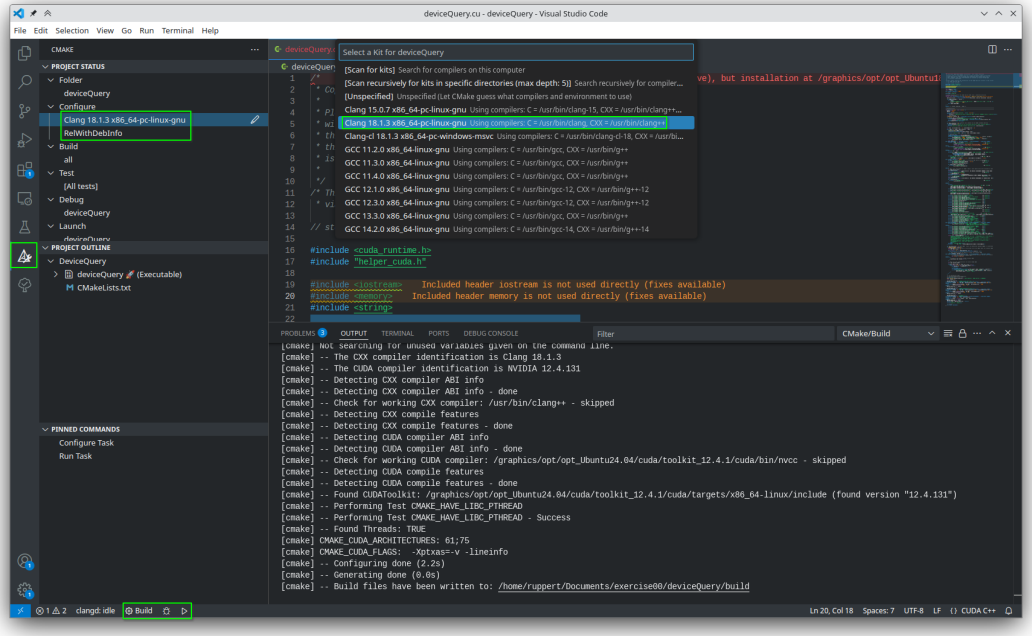
- CLion

  Supports CUDA. Only free to use with a student's license – you're on your own.

If you are unsure about what to choose, try using VSCode as shown below.

0.4 Import, Compile and Run a Sample CUDA Program

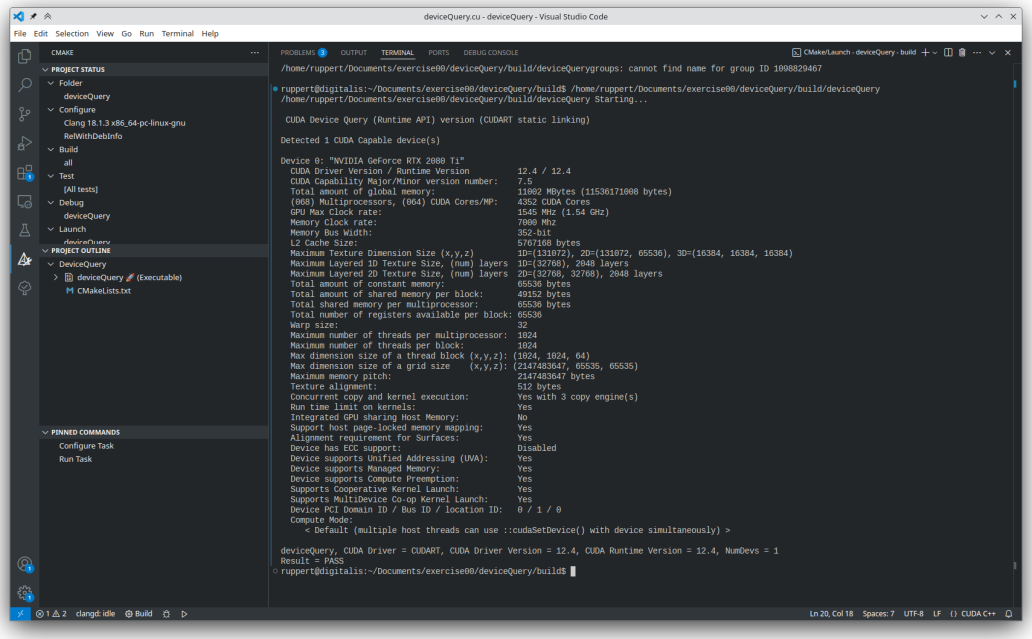
Open the `deviceQuery` sample application folder from exercise00 in VSCode. See the following figures for details on how to configure, compile, and run the project.



To compile and run the project, follow these steps:

- Select a compiler. (Any one of them should work, simply use the latest version of GCC/Clang.)
- In the CMake settings, set the build variant to "RelWithDebInfo".
  (The debug mode can run significantly slower for some projects.)
- Build the project using the status bar icon.
- Run the project using the status bar icon.

You should get the following output:



0.4.1 Using only SSH

If you prefer not using VNC and just want to use SSH instead, you can also compile and run the `deviceQuery` application by running the following commands in the project folder:

```
# this should work for all CMake-based projects
mkdir build
cd build
cmake ..
make
```

Then, run the compiled program using `./deviceQuery`.